

Formato delle variabili

I nomi delle variabili sono dichiarate con `let`

- Possono contenere lettere, numeri e i caratteri `(.)` e `(')`
- devono iniziare con lettera minuscola o con `_`

esempi corretti:

[3]: `let x = 3 + 4;`

[3]: `val x: int = 7`

[4]: `let x_plus_3 = x + 3`

[4]: `val x_plus_3: int = 10`

[5]: `let x' = x + 1`

[5]: `val x': int = 8`

esempio errato

[6]: `let SeveM = 3 + 4`
 ↓
 maiusc

// errore S è maiuscola

[7]: `let 7x = 7`

tutto è un'espressione

- Coerentemente con il paradigma funzionale, che non prevede uno stato del programma modificabile, i costrutti fondamentali sono le espressioni
- le espressioni possono contenere dichiarazioni (o meglio "bindings")

tipi di base

- OCaml è un linguaggio fortemente tipato
- I tipi base sono i seguenti
- int: numeri interi
- float: numeri frazionari: floating point a doppia precisione
- bool: true o false
- char: singoli caratteri
- string: stringhe (racchiuse tra virgolette: "abcd")
- unit: tipo usato in casi particolari (simile a void in altri linguaggi), prevede come unico valore ()

operazioni su int e float

operazioni su numeri frazionari (float) si scrivono diversamente da quelle su interi (int)

- Si scrivono seguite da un punto
+, -, *, /.

operazioni su interi:

3 + 5;;

operazioni su float

3.2 +. 5.2

- tipi operandi deve essere coerente con quello delle operazioni, altrimenti il compilatore segnala errore

conversione dei tipi da int e float

- non sono automatiche le conversioni
- esempio 3 non può essere utilizzato al posto di 3.0

si deve usare:

- float - of - int
- int - of - float

esempio

[11]: float_of_int 3;

int_of_float 3.0;

[11]: float = 3.

int = 3

OPERAZIONI SU Bool

simbassi

[13] let x = bool

[13]: val x: bool = true

x && false

x || false

not x

Attenzione

comparando si fa com = non com == !

Per vedere se due espressioni sono diverse si usa <>

OPERAZIONI su char e string

[16]: `let c = 'a';`
`int_of_char c;` (esiste anche `char_of_int`)

[16]: `val c: char = 'a'`
`-: int = 97`

Concatenazione

[17]: `let h = "hello"`
`let hw = h ^ "world" // si concatena con ^`

[17] `val h: string = "hello"`

`val hw: string = "helloworld"`

[18]: `int_of_string "10"`

[18]: `-: int = 10`

I tipi tuple

- è una sequenza di valori separata da virgole

`let t = (10, "hello", 12.5)`

- i tipi tuple sono il prodotto degli elementi che li compongono

esempi

immaginiamo che `int` e `float` corrispondano agli insiemi $\mathbb{Z}$ e $\mathbb{R}$

il tipo `int * float` corrisponde all'insieme $\mathbb{Z} \times \mathbb{R}$

esempi

[20]: `let p = (3.0, 5.0)` // le parentesi non sono necessarie

[20]: `val p: Float * Float = (3., 5.)`

- Ha tipo `float * float`

- la cosa interessante è che possiamo destrutturare una tupla

[21]: `let (x, y) = p` ;

`x +. y` ;

[21]: `val x: float = 3`

`val y: float = 5`

`-. float = 8`

Funzioni

In OCaml le espressioni sono valori per rispettare il vincolo funzionale

- Partendo con λ -calcolo (con espressioni su interi)

si riprende quanto visto con λ calcolo

Per esempio $\lambda x. x + 1$ è in OCaml

[22]: `fun x -> x + 1`

[22]: `-: int -> int = <fun>`

e la sua applicazione è $(\lambda x. x + 1) 10$

[23]: `(fun x -> x + 1) 10` ;

[23]: `-: int = 11`

Osservazioni

- 1° nell'esempio con la def di funzione, l'interprete ha restituito un output simile a quello che avevamo visto negli esempi di espressioni mostrati in precedenza:
- ABBIAMO IL SIMBOLO $-$, che denota che non è stato effettuato un binding (non abbiamo dichiarato una variabile)
- ABBIAMO IL TIPO dell'espressione, in questo caso $\text{int} \rightarrow \text{int}$ che prevede un intero come parametro e restituisce un intero
- DOPO = ABBIAMO IL RISULTATO della valutazione dell'espressione che generalmente è $\langle \text{fun} \rangle$, visto che non è possibile per l'interprete rappresentare i valori funzione

Funzioni: altri esempi del λ -calcolo

$$\bullet \lambda x. \lambda y. x + y$$

$$[24]: \text{Fun } x \rightarrow \text{Fun } y \rightarrow x + y$$

$$[24] \quad \therefore \text{int} \rightarrow \text{int} \rightarrow \text{int} = \langle \text{fun} \rangle$$

$$\bullet \lambda f. \lambda m. f m + m$$

$$[25]: \text{Fun } f \rightarrow \text{Fun } m \rightarrow f m + m$$

$$[25]: \therefore ('a \rightarrow \text{int}) \rightarrow 'a \rightarrow \text{int} = \langle \text{fun} \rangle$$

Type inference

- 1° espressione: $\text{fun } x \rightarrow \text{fun } y \rightarrow x + y$ è una funzione che:
- Prende x e restituisce $\text{fun } y \rightarrow x + y$
- Prende y e restituisce il risultato di $x + y$

$$\begin{array}{ccccccc} \text{int} & \rightarrow & (\text{int} & \rightarrow & \text{int}) & = & \text{visto che } \rightarrow \text{ è associativa} \\ \downarrow & & \downarrow & & \downarrow & & = \text{int} \rightarrow \text{int} \rightarrow \text{int} \\ x & & y & & \text{risultato} & & \text{A destra si possono togliere le parentesi} \end{array}$$

- nella seconda funzione si applica allo stesso modo $\text{fun } f \rightarrow \text{fun } m \rightarrow f\ m + m$
 m si deduce sia int perché operando del $+$
 mentre f è una funzione a cui viene applicato $m:\text{int}$ e il risultato di $f\ m$ deve
 essere int per sommarlo a m

$(\text{int} \rightarrow \text{int}) \rightarrow \text{int} \rightarrow \text{int}$

Alcuni esempi

$(\lambda f. \lambda m. f\ m + 1)(\lambda x. 2x)\ 10$

[26]: $(\text{fun } f \rightarrow \text{fun } m \rightarrow f\ m + 1)(\text{fun } x \rightarrow 2 * x)\ 10$

[26]: $-\text{int} = 21$

- questo è un esempio di funzione **high-order** (di ordine superiore), ossia una funzione che
 prende un'altra funzione (f) come parametro e la usa nel proprio corpo

- il valore di Ocaml è lo stesso che si ottiene facendo la β -riduzione

$(\lambda f. \lambda m. f\ m + 1)(\lambda x. 2x)\ 10$
 $\rightarrow (\lambda m. (\lambda x. 2x)\ m + 1)\ 10$
 $\rightarrow (\lambda m. 2m + 1)\ 10$
 $\rightarrow 2 \cdot 10 + 1 = 21$

Funzioni con + parametri

- il costrutto fun può avere più di un parametro

$(\text{fun } x\ y \rightarrow x + 1)\ 3\ 4$

equivalente a $(\text{fun } x \rightarrow \text{fun } y \rightarrow x + 1)\ 3\ 4$

un altro modo di definire le funzioni

si può utilizzare let

- let somma = fun x y -> x + y

si può semplicemente scrivere

- let somma x y -> x + y

somma 3 4

si può definire anche con function

es: function x -> x + 1 // però function prevede un solo argomento

non esiste il return

- nella programmazione imperativa con i comandi ha senso
- nella programmazione funzionale con sole espressioni no